

arithmetic

CSCI
134

expression tree



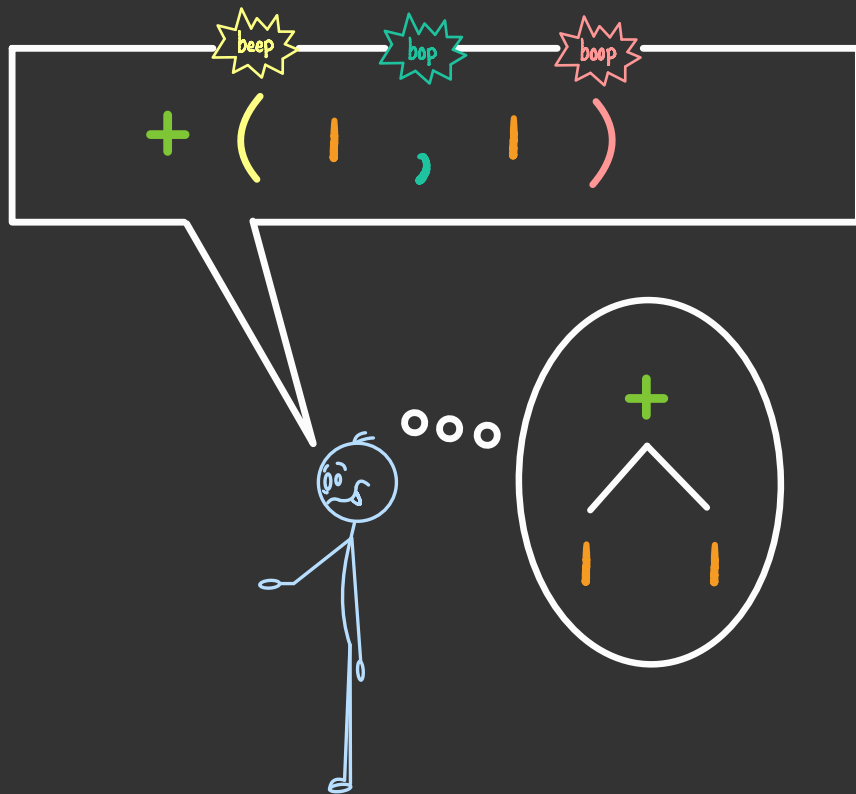
how do we ask python to compute $1 + 1$?

python's protocol

type the label at the root,
then if the root has children:

- ▶ type an open paren
- ▶ communicate the first child using the protocol
- ▶ while more children remain:
 - ▶ type a comma
 - ▶ communicate the next child using the protocol
- ▶ type a closing paren

like this?

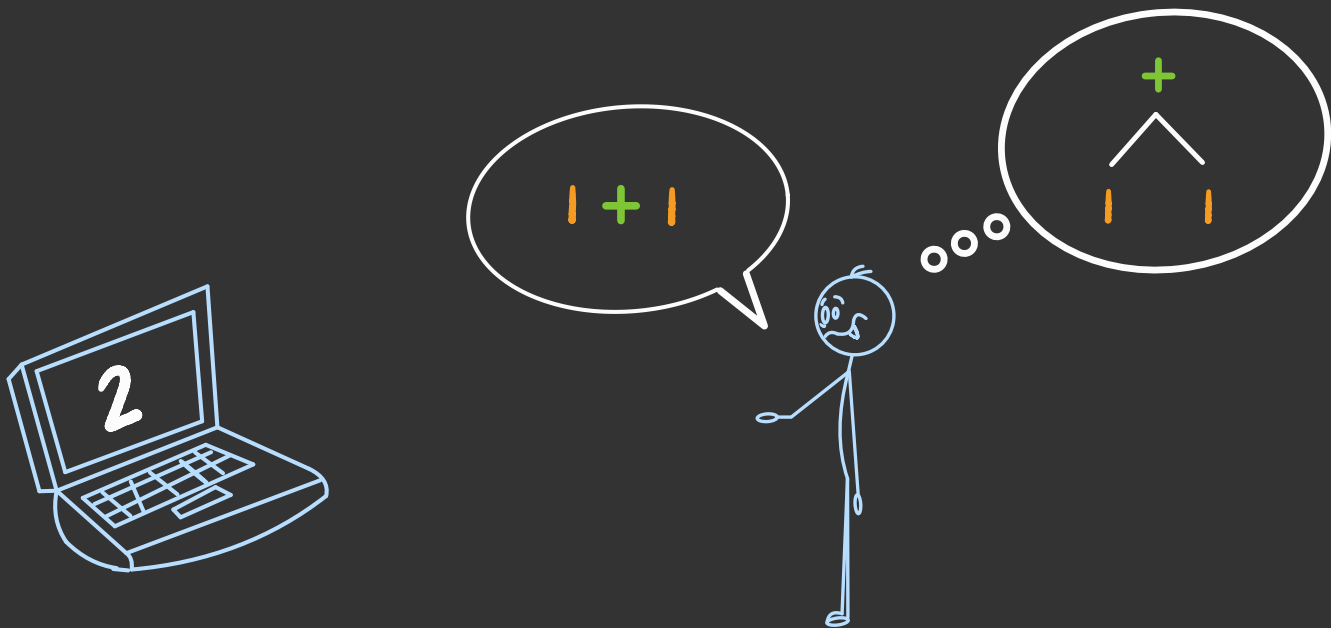


```
In [1]: +(1, 1)
```

```
TypeError                                Traceback (most recent call last)  
Cell In[1], line 1  
----> 1 +(1, 1)
```



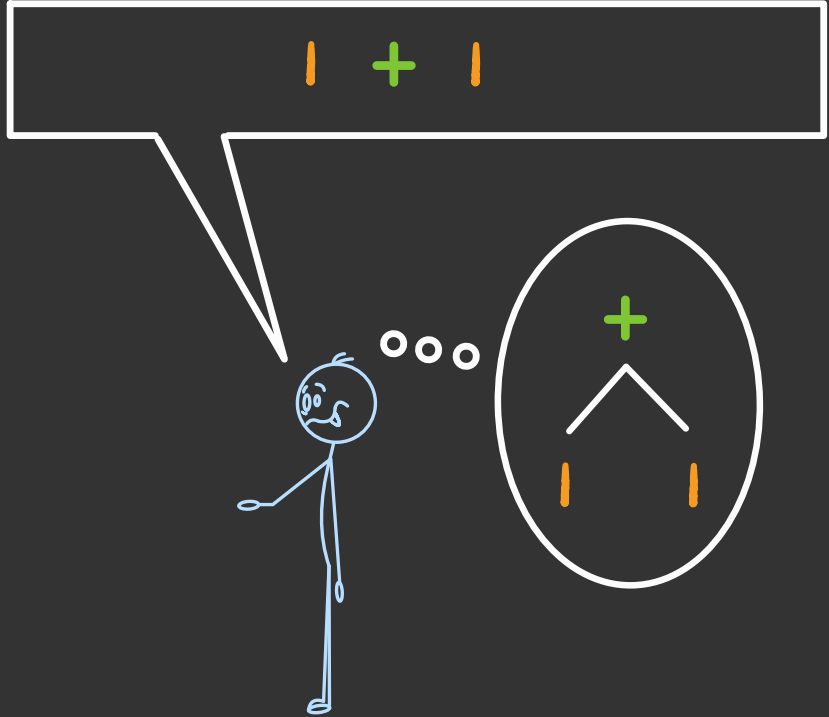
no, not like that



like this

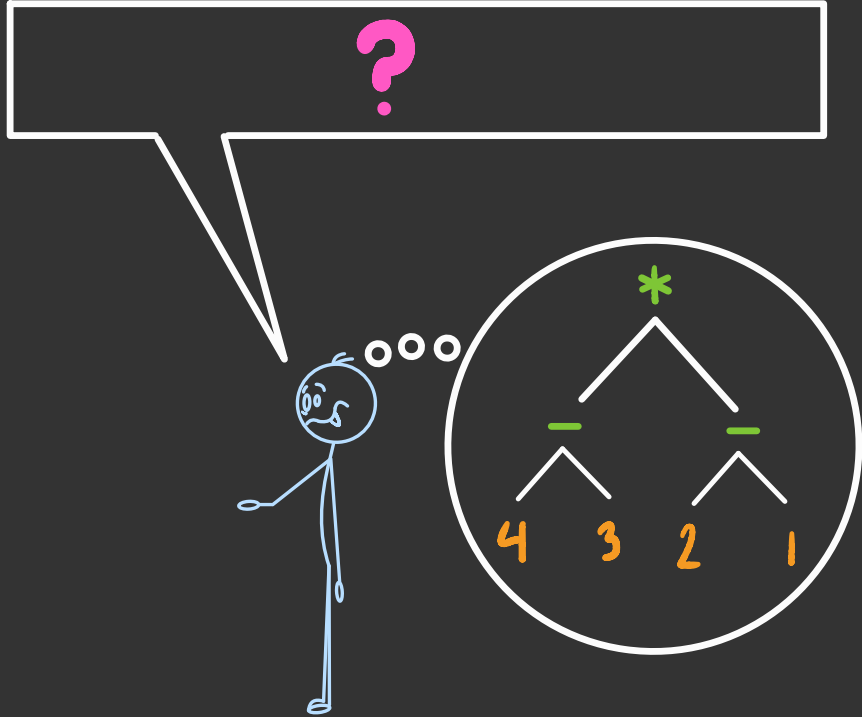
infix protocol

- ▶ if not at the root, **type an open paren**
- ▶ communicate the first child using the **protocol**
- ▶ type the current node's label
- ▶ communicate the second child using the **protocol**
- ▶ if not at the root, **type a closing paren**



infix protocol

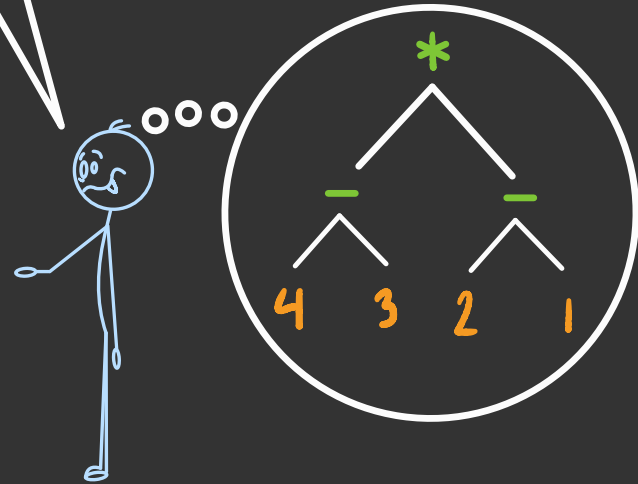
- ▶ if not at the root, type an open paren
- ▶ communicate the first child using the protocol
- ▶ type the current node's label
- ▶ communicate the second child using the protocol
- ▶ if not at the root, type a closing paren



infix protocol

- ▶ if not at the root, **type an open paren**
- ▶ communicate the first child using the **protocol**
- ▶ type the current node's label
- ▶ communicate the second child using the **protocol**
- ▶ if not at the root, **type a closing paren**

(4 - 3) * (2 - 1)



infix protocol

- ▶ if not at the root, type an open paren
- ▶ communicate the first child using the protocol
- ▶ type the current node's label
- ▶ communicate the second child using the protocol
- ▶ if not at the root, type a closing paren

python uses the **infix protocol** for the following arithmetic functions:

+ addition

- subtraction

***** multiplication

/ division

// integer division

% modulo

```
In [1]: 18 + 4  
Out[1]: 22
```

```
In [2]: 18 - 4  
Out[2]: 14
```

```
In [3]: 18 * 4  
Out[3]: 72
```

```
In [4]: 18 / 4  
Out[4]: 4.5
```

```
In [5]: 18 // 4  
Out[5]: 4
```

```
In [6]: 18 % 4  
Out[6]: 2
```

$$4 \overline{) 30} \quad ?^R ?$$

python uses the **infix** protocol for the following arithmetic functions:

+ addition

- subtraction

***** multiplication

/ division

// integer division

% modulo

```
In [1]: 30 + 4  
Out[1]: 34
```

```
In [2]: 30 - 4  
Out[2]: 26
```

```
In [3]: 30 * 4  
Out[3]: 120
```

```
In [4]: 30 / 4  
Out[4]: 7.5
```

```
In [5]: 30 // 4  
Out[5]: 7
```

```
In [6]: 30 % 4  
Out[6]: 2
```

integer
division

modulo

7 R 2

4 | 30

python uses the **infix**
protocol for the following
arithmetic functions:

+ addition

- subtraction

* multiplication

/ division

// integer division

% modulo

```
In [1]: 30 + 4  
Out[1]: 34
```

```
In [2]: 30 - 4  
Out[2]: 26
```

```
In [3]: 30 * 4  
Out[3]: 120
```

```
In [4]: 30 / 4  
Out[4]: 7.5
```

```
In [5]: 30 // 4  
Out[5]: 7
```

```
In [6]: 30 % 4  
Out[6]: 2
```

examples

we cannot use the
prefix protocol for
arithmetic operators

```
In [1]: +(1, 1)
-----
TypeError
<ipython-input-1-4354ba4905b5> in <module>
----> 1 +(1, 1)

TypeError: bad operand type for unary +: 'tuple'

In [2]: 1 + 1
Out[2]: 2

In [3]: ((1+2)*3) + (4 / (5-6))
Out[3]: 5.0

In [4]: type(((1+2)*3) + (4 / (5-6)))
Out[4]: float

In [5]: 5-6
Out[5]: -1

In [6]: type(5-6)
Out[6]: int

In [7]: 4/(5-6)
Out[7]: -4.0

In [8]: 4/-1
Out[8]: -4.0
```

the division operator
always produces a float

examples

the **integer division** operator
always produces an **int**

the **modulo** operator
always produces an **int**

```
In [10]: 5/2
Out[10]: 2.5

In [11]: type(5/2)
Out[11]: float

In [12]: 4/2
Out[12]: 2.0

In [13]: 4//2
Out[13]: 2

In [14]: 5//2
Out[14]: 2

In [15]: 60/5
Out[15]: 12.0

In [16]: 62//5
Out[16]: 12

In [17]: 62%5
Out[17]: 2

In [18]: 5 % 2
Out[18]: 1

In [19]: 6 % 2
Out[19]: 0
```

examples

the **addition** operator
can also glue two
strs together
(concatenation)

a **str** can be
multiplied
by an **int**

```
In [20]: "hello" + "there"
```

```
Out[20]: 'hellothere'
```

```
In [21]: "hello" + " there"
```

```
Out[21]: 'hello there'
```

```
In [22]: "hello" + " " + "there"
```

```
Out[22]: 'hello there'
```

```
In [23]: "hello" * 3
```

```
Out[23]: 'hellohellohello'
```

```
In [24]: "hello" * 5
```

```
Out[24]: 'hellohellohellohellohello'
```

```
In [25]: "hello " * 5
```

```
Out[25]: 'hello hello hello hello hello '
```

examples

if parens are missing,
python will infer the
expression tree based
on standard order of
operations

```
In [26]: ((1+2)+3)
```

```
Out[26]: 6
```

```
In [27]: 1+2+3
```

```
Out[27]: 6
```

```
In [28]: ((2*3)+4)
```

```
Out[28]: 10
```

```
In [29]: 2*3+4
```

```
Out[29]: 10
```

```
In [30]: (4 + (2*3))
```

```
Out[30]: 10
```

```
In [31]: 4+2*3
```

```
Out[31]: 10
```

```
In [32]: (4+2)*3
```

```
Out[32]: 18
```